INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE OCCIDENTE

BASES DE DATOS NO RELACIONALES

ITESO, Universidad Jesuita de Guadalajara

Multi-Database Backend for a Religious / Wellness Application

Multi_NRDB_Implementation

Name and Student ID of each team member

Luis Mariano Romo Medina — 758007

Germán González Partida — 759466

Santiago Ismael López Contreras — 744857

Teacher: Leobardo Ruiz Rountree

Date: 15/05/2026

Content

Table of Contents

Project Name ***4***

Introduction ***4***

Project Scope ***5***

Functional Requirements ***6***

User management (MongoDB) 15
Content and interactions (MongoDB) 15
Social graph and recommendations (Dgraph) 15
Activity analytics (Cassandra) 15
Semantic retrieval (ChromaDB) 15
Operational requirements 15

Design ***16***

Data Models Cassandra **16**

Table: activity_by_user 16
Table: activity_by_day 17
Table: activity_by_content 18
Justification 18

MongoDB Data Models **19**

Collection: users 19
Collection: Content 20
Collection: comments 21
Collection: Content Likes 21
Collection: Internal Shares 22
Collection: External Shares 22
Collection: notes 23
Design justification 23

Data Model Dgraph **24**

Data Models Overall **28**

MongoDB — Operational document store 28
Cassandra — Analytics store 28
Dgraph — Graph store 29
ChromaDB — Vector store 30

Queries **32**

Indexes and Optimizations **38**

MongoDB indexes 38
MongoDB aggregation as an optimization 38
Cassandra partition and clustering keys 38
Dgraph schema indexes 38
ChromaDB precomputed embeddings 39

Conclusion ***40***

Sources of Information ***41***

Appendix ***42***

Final Demo Flow **42**

Terminal A — services 42

Terminal B — demo 42

Known Limitations **43**

Project Name

Multi-Database Backend for a Religious / Wellness Application (internal repository name: Multi_NRDB_Implementation).

The project is a backend that supports a religious/wellness mobile and CLI client. It combines four non-relational database which are MongoDB, Apache Cassandra, Dgraph, and ChromaDB and each one of them are used only for the query patterns it is best suited to handle. User-facing flows are exposed through a Falcon HTTP API, while analytics operations are issued through dedicated command-line entry points.

Introduction

The objective of this project was clear; we wanted to design and implement a project of interest that was based on non-relational databases whose access patterns are not uniform and are unique to the kind of query needed. The application supports user accounts, profile preferences, content (prayers, devotionals, and similar items), personal notes, comments, likes, internal and external shares, follow relationships between users, event discovery, activity analytics, and semantic search over religious and wellness content.

A single database paradigm would not fit all these requirements with the same quality or would be unnecessarily complex in comparison to using NoSQL. Document-oriented data, time-series analytics, graph relationships, and semantic similarity each have characteristics that map naturally to a different functional requirement that were exposed in previous deliveries. For this reason, the system uses MongoDB as the operational document store, Cassandra as the analytics store, Dgraph for relationship traversal, and ChromaDB for vector retrieval.

Each database was selected and modeled by first identifying the queries that need to be answered and then choosing the storage paradigm and schema that minimizes the cost of those queries. In the previous delivery we already laid down the bases to identify for each Functional Requirement, the database needed to support that, the result is one coherent backend in which each database has a clear, narrow responsibility.

Project Scope

The implemented scope of this project includes:

- User registration, login, profile retrieval, and local CLI session handling.
- MongoDB document operations for users, content, notes, comments, likes, internal shares, and external shares.
- MongoDB indexes on the fields used by the operational queries, created during the seed step.
- A MongoDB aggregation pipeline that joins content with likes and comments to produce engagement statistics.
- Dgraph graph relationships for follows, interested_in, attends, and event_topic, including reverse edges used for counts and recommendations.
- Dgraph queries for graph summaries, user-to-user recommendations (by shared interest and by location), local event discovery, and event recommendations.
- Cassandra analytics tables activity_by_user, activity_by_day, and activity_by_content, designed around specific analytics queries and accessed through their primary keys.
- ChromaDB semantic search, preference-based content recommendation, RAG-style context retrieval, and a local template-based RAG answer.
- A single seed command that loads consistent demo data into MongoDB, Dgraph, and ChromaDB and ensures that the same identifier is used for a given user across MongoDB and Dgraph.
- A Docker Compose configuration that starts with MongoDB, Dgraph, and Cassandra; ChromaDB persists locally on disk.

The following listed points are intentionally left out of the project either because they fall out of our original scope or because of lack of time, the reasons of why they were left out or partially implemented are explained in the conclusions and the Known Limitations section of the document.

- Synchronization of user preferences between MongoDB and Dgraph is limited to graph-relevant user fields. Preference removal does not delete the corresponding interest edges in Dgraph.
- Cassandra analytics data is written through explicit demo commands (log_session, log_activity) rather than through every normal application action.
- The rag_answer command produces a local, template-based answer composed from retrieved Chroma documents. It does not call an external Large Language Model.
- Some CLI commands (update, add_pref, rem_pref) are interactive.

Functional Requirements

FR-01: User Registration

Requirement: The system shall allow users to register through a command-line interface by providing a unique email, a valid password, and at least one preference.

Test Case: User executes register --email test@mail.com --password 1234 –preferences prayer

Expected Result: User account is successfully created.
Validation:
- Email is unique
- Password is stored securely
- Preferences are stored correctly

FR-02: User Login

Requirement: The system shall allow users to log in through a command-line interface using valid credentials.

Test Case: login --email test@mail.com --password 1234

Expected Result: User is authenticated successfully.
Validation:
- Email exists
- Password matches
- Session is created

FR-03: Update User Information

Requirement: The system shall allow authenticated users to update personal information such as age, location and preferences.

Test Case: update-user --field location --value Guadalajara

Expected Result: User information is updated.
Validation:
- User is authenticated
- Field is valid
- Value is correctly stored

FR-04: Manage Preferences

Requirement: The system shall allow users to add or remove preferences.
Test Case: add-preference --preference prayer

Expected Result: Preference is updated.
Validation:

- Preference is valid
- No duplicates
- Changes persist

FR-05: Session Tracking

Requirement: The system shall track login and logout events.
Test Case: User logs in and logs out.

Expected Result: Events are recorded.
Validation:

- Events stored with timestamp
- Stored in Cassandra

FR-06: Create Content (Admin)

Requirement: The system shall allow administrators to create content.
Test Case: create-content --type text --title "Hope"

Expected Result: Content is created.
Validation:

- Admin privileges verified
- Content stored correctly

FR-07: Like Content

Requirement: The system shall allow authenticated users to like content.
Test Case: like-content --id 123

Expected Result: Like is recorded.
Validation:
- User authenticated
- No duplicate likes
- Stored in Mongo + logged in Cassandra

FR-08: Comment on Content

Requirement: The system shall allow users to comment on content.
Test Case: comment --content-id 123 --text "Nice"

Expected Result: Comment is created.
Validation:
- Content exists
- Comment stored
- Logged in Cassandra

FR-09: Get Comments by Content

Requirement: Retrieve all comments for content.
Test Case: comments-by-content --content-id 123

Expected Result: List of comments returned.
Validation:
- Correct content_id
- Structured output

FR-10: Get Comments by User

Requirement: Retrieve user's comments.
Test Case: comments-by-user

Expected Result: List returned.
Validation:
• Only user's comments
• User authenticated

FR-11: Follow Users

Requirement: Users can follow other users.
Test Case: follow-user --user-id 456

Expected Result: Relationship created.
Validation:
• No self-follow
• No duplicates
• Stored in Dgraph

FR-12: Prevent Duplicate Follows

Requirement: The system shall prevent duplicate follow relationships.
Test Case: Follow same user twice.

Expected Result: Second attempt rejected.
Validation:
• No duplicate edge created

FR-13: Share Content (Internal)
Requirement: Users can share content with other users.
Test Case: share-content --content-id 123 --user-id 456

Expected Result: Share recorded.
Validation:
• Stored in Mongo
• Logged in Cassandra

FR-14: Recommend Users (Graph)

Requirement: Recommend users based on connections and preferences.
Test Case: recommend-users

Expected Result: List returned.
Validation:
• Graph traversal
• No duplicates
• Not already followed

FR-15: Recommend Users by Location

Requirement: Recommend users based on location.
Test Case: recommend-users-by-location

Expected Result: Users returned.
Validation:
• Same location
• Excludes self

FR-16: Retrieve Activity History

Requirement: Users can retrieve activity history.
Test Case: get-activity-history

Expected Result: List returned.
Validation:
• Data from Cassandra
• Ordered by time

FR-17: Filter Activity History

Requirement: Filter activity by date/type.
Test Case: get-activity-history --type like

Expected Result: Filtered results.
Validation:
• Filters applied correctly

FR-18: External Sharing Tracking

Requirement: Track external shares.
Test Case: share-external --platform twitter

Expected Result: Share recorded.
Validation:
• Platform valid
• Stored in Mongo + Cassandra

FR-19: Activity Logging

Requirement: Log user actions automatically.
Test Case: User performs actions.

Expected Result: Logs created.
Validation:
• Stored in Cassandra
• Includes metadata

FR-20: Daily Active Users

Requirement: Calculate daily active users.
Test Case: get-daily-active-users --date

Expected Result: Count returned.
Validation:
• Unique users counted

FR-21: Create Notes

Requirement: Users can create notes.
Test Case: create-note --text "..."

Expected Result: Note created.
Validation:
• Stored in Mongo
• Linked to user

FR-22: Retrieve Notes

Requirement: Users can retrieve notes.
Test Case: get-notes

Expected Result: Notes returned.
Validation:
• Ordered by timestamp

FR-23: Update/Delete Notes

Requirement: Users can update/delete notes.
Test Case: delete-note --id

Expected Result: Note removed.
Validation:
• Ownership verified

FR-24: Recommend Content
Requirement: Recommend content based on preferences.
Test Case: recommend-content

Expected Result: Content returned.
Validation:
• Tag matching

FR-25: Local Events

Requirement: Retrieve local events.
Test Case: get-local-events

Expected Result: Events returned.
Validation:
• Location match
• Future events

FR-26: Attend Event

Requirement: Users can attend events.
Test Case: attend-event --event-id

Expected Result: Relationship created.
Validation:
• Stored in Dgraph

FR-27: Recommend Events

Requirement: Recommend events via connections.
Test Case: recommend-events-by-connections

Expected Result: Events returned.
Validation:
- Graph traversal

FR-28: Chatbot Interaction

Requirement: Users can interact with chatbot.
Test Case: chatbot --prompt

Expected Result: Response returned.
Validation:
- Prompt processed

FR-30: Semantic Search

Requirement: Perform semantic search.
Test Case: search-content --query

Expected Result: Relevant content returned.
Validation:
- Uses embeddings (Chroma)

FR-31: RAG Response Generation

Requirement: Generate responses using retrieved content.
Test Case: Chatbot prompt

Expected Result: Contextual response.
Validation:
- Uses retrieval + generation

FR-32: Content Metrics

Requirement: Provide engagement metrics.
Test Case: stats-by-content

Expected Result: Metrics returned.
Validation:
- Aggregated from Cassandra

FR-33: System Metrics

Requirement: Provide system-wide metrics.
Test Case: get-system-stats

Expected Result: Aggregated data.
Validation:
- Global aggregation

FR-34: Trending Content

Requirement: Identify trending content.
Test Case: trending-content

Expected Result: Top content returned.
Validation:
- Engagement score

FR-35: Scalable Logging

Requirement: Support scalable activity log storage.
Test Case: Simulate multiple actions.

Expected Result: Logs stored successfully.
Validation:
- No data loss
- High write performance

The functional requirements addressed by this project, grouped by area, are listed below.

User management (MongoDB)

The system must allow user registration and login, store user profiles including email, username, location, and preferences, and retrieve a user's own profile after login.

Content and interactions (MongoDB)

The system must store content items, personal notes, comments, likes, internal shares (user-to-user), and external shares (to external platforms). It must support manual references between users and content using ObjectIds and embed small display fields (such as username or content title) where they are needed to avoid extra reads on display paths.

Social graph and recommendations (Dgraph)

The system must represent users, events, and interests as nodes and connect them with follows, attends, interested_in, and event_topic edges. It must support counting followers and event attendees through reverse traversal, recommending users by shared interests, recommending users in the same location, finding local events, and recommending events based on a user's interests.

Activity analytics (Cassandra)

The system must record user activity events and answer the following analytics queries by primary key: activity history of a specific user, daily activity across the system, and engagement metrics for a specific content item. The same event is written to several tables so that each query reads from a partition shaped for it.

Semantic retrieval (ChromaDB)

The system must embed content items (title, type, tags, and text) using the all-MiniLM-L6-v2 model and answer the following queries: semantic search by free-text query, preference-based content recommendation, retrieval of semantic context for a query, and composition of a local template-based answer that cites the retrieved sources.

Operational requirements

The system must be reproducible through docker compose for the server-side databases and through a single seed command for demo data. Identifiers between MongoDB and Dgraph must be consistent so that a user found in MongoDB can also be reached in the grap

Design

Data Models Cassandra

The Cassandra schema was designed following query-based modeling principles. This means that tables were created based on the questions the system needs to answer, rather than trying to avoid data duplication.

Three main tables were designed to support activity tracking and analytics:

- activity_by_user
- activity_by_day
- activity_by_content

Each table is optimized for a specific access pattern.

Table: activity_by_user

This table is used for user activity history and filtered history queries.

Query Supported: Get activity by user_id ordered by timestamp.
Primary Key: PRIMARY KEY ((user_id), timestamp, activity_id)

Explanation:

- The **partition key (user_id)** groups all activity events of a specific user into a single partition.
- The **clustering key (timestamp, activity_id)** ensures that events are stored in chronological order from more recent events first.
- This allows efficient retrieval of a user's activity history without requiring additional sorting.

Table structure:

```
activity_by_user (
        user_id       uuid,
        user_ref      text,
        timestamp     timestamp,
        activity_id   uuid,
        activity_type text,
        content_id    uuid,
        content_ref   text,
        metadata      text,
        PRIMARY KEY ((user_id), timestamp, activity_id)
    ) WITH CLUSTERING ORDER BY (timestamp DESC);
```

Table: activity_by_day

This table supports daily activity analytics such as:

- daily active users
- total events for a given day
- activity breakdown by type
- trending content by day

Query Supported: Get activity by date ordered by timestamp.
Primary Key: PRIMARY KEY ((date), timestamp, activity_id)

Explanation:

- The **partition key (date)** groups all events occurring on the same day.
- The **clustering key (timestamp)** orders events chronologically within that day.
- This structure enables efficient daily analytics queries, such as calculating daily active users, or calculate trending content.

Table structure:

```
activity_by_day (
        date          date,
        timestamp     timestamp,
        activity_id   uuid,
        user_id       uuid,
        user_ref      text,
        activity_type text,
        content_id    uuid,
        content_ref   text,
        PRIMARY KEY ((date), timestamp, activity_id)
    ) WITH CLUSTERING ORDER BY (timestamp DESC);
```

Table: activity_by_content

This table is used to compute engagement metrics such as total interactions and breakdown by activity type for a content item.

Query Supported: Get activity by content_id ordered by timestamp.
Primary Key: PRIMARY KEY ((content_id), timestamp, activity_id)

Explanation:

- The **partition key (content_id)** groups all interactions related to a specific content item.
- The **clustering key (timestamp)** ensures chronological ordering of events.
- This supports queries related to content engagement metrics.

Table structure:

```
activity_by_content (
        content_id    uuid,
        content_ref   text,
        timestamp     timestamp,
        activity_id   uuid,
        user_id       uuid,
        user_ref      text,
        activity_type text,
        metadata      text,
        PRIMARY KEY ((content_id), timestamp, activity_id)
    ) WITH CLUSTERING ORDER BY (timestamp DESC);
```

Justification

The design reflects Cassandra's denormalized and query-driven nature. Instead of storing activity in a single generic table, the schema duplicates data across multiple tables so each query can be answered efficiently using its primary key.

Each table is designed to support a specific query pattern, avoiding expensive operations such as joins or full table scans.

This design ensures:

- high write throughput for activity logging
- efficient reads for user history, daily analytics, and content metrics
- scalability for large event volumes
- predictable query performance without joins

MongoDB Data Models

MongoDB is used to store flexible and user-centered data within the application.

The following collections were designed to support the functional requirements:

- users
- content
- comments
- shares
- notes

Each collection is structured to support specific operations such as creation, retrieval, and updates, while indexes are applied to optimize performance.

Collection: users

Description: Stores user profile information, authentication data, and preferences.

Document Structure

```
{
    "_id": ObjectId,
    # Core identity
    "username": str,
    "email": str, # UNIQUE INDEX
    "password_hash": str,
    "age": int,
    "location": str,
    # Attribute Pattern
    "preferences": [
        {
            "name": str,
            "category": str, # prayer, meditation, bible, etc
            # "added_at": datetime
        }
    ],
    "last_login": datetime,
    "created_at": datetime,
    "updated_at": datetime,
    # Only RECENT interactions embedded
    "recent_likes": [
        {
            "content_id": ObjectId,
            "title": str,
            # "liked_at": datetime
        }
    ]
}
```

Indexes

```
db.users.createIndex({ email: 1 }, { unique: true })
db.users.createIndex({ location: 1 })
```

Justification: Indexes improve performance for login operations and location-based recommendations.

Collection: Content

Description: Stores content created by the users, including metadata and categorization.
Document Structure

```
{
    "_id": ObjectId,
    # Content info
    "title": str,
    "type": str, # text, image, audio, etc

    # Creator reference
    "created_by": {
        "user_id": ObjectId,
        "username": str
    },
    # Metrics
    "likes_count": int,
    "comments_count": int,
    "shares_count": int,
    "recent_likes": [
        {
            "user_id": ObjectId,
            "username": str,
            # "liked_at": datetime
        }
    ],
    "recent_comments": [
        {
            "comment_id": ObjectId,
            "user_id": ObjectId,
            "username": str,
            "text": str,
            # "created_at": datetime
        }
    ],

    "created_at": datetime
}
```

Indexes

```
db.content.createIndex({ tags: 1 })
db.content.createIndex({ created_at: -1 })
```

Justification: Indexes support content recommendations and retrieval of recent content.

Collection: comments

Description: Stores user comments associated with content.
Document Structure

```
comments_types = {
    "_id": ObjectId,
    "content": {
        "content_id": ObjectId,
        "title": str
    },
    "user": {
        "user_id": ObjectId,
        "username": str
    },
    "text": str,
    "created_at": datetime
}
```

Indexes

```
db.comments.createIndex({ content_id: 1 })
db.comments.createIndex({ user_id: 1 })
db.comments.createIndex({ created_at: -1 })
```

Justification: Indexes allow efficient retrieval of comments by content and by user.

Collection: Content Likes

Description: Stores users' likes associated with content.
Document Structure

```
likes_types = {
    "_id": ObjectId,

    "user": {
        "user_id": ObjectId,
        "username": str
    },

    "content": {
        "content_id": ObjectId,
        "title": str
    },

    "created_at": datetime
}
```

Collection: Internal Shares

Description: Stores internal sharing events
Document Structure

```
shares_types = {
    "_id": ObjectId,

    "from_user": {
        "user_id": ObjectId,
        "username": str
    },

    "to_user": {
        "user_id": ObjectId,
        "username": str
    },

    "content": {
        "content_id": ObjectId,
        "title": str
    },

    "created_at": datetime
}
```

Justification: Indexes enable efficient querying for analytics and user interactions.

Collection: External Shares

Description: Stores external sharing events
Document Structure

```
external_shares_types = {
  "_id": ObjectId,

  "user": {
    "user_id": ObjectId,
    "username": str
  },

  "content": {
    "content_id": ObjectId,
    "title": str
  },

  "platform": str,  # Twitter, Facebook, Insta
  "created_at": datetime
}
```

Collection: notes

Description: Stores private notes created by users.
Document Structure

```
notes_types = {
    "_id": ObjectId,

    "user": {
        "user_id": ObjectId,
        "username": str
    },

    "title": str,
    "text": str,

    "created_at": datetime,
    "updated_at": datetime
}
```

Indexes

```
db.notes.createIndex({ user_id: 1 })
db.notes.createIndex({ created_at: -1 })
```

Justification : Indexes support fast retrieval of notes and sorting by recency.

Design justification

MongoDB is used to store structured but flexible data such as user profiles, content, and interactions. Indexes improve performance for frequent queries like the login, recommendations, and retrieving user-specific data.

Data Model Dgraph

Dgraph is used in the system to represent and query relationship-based information that is not as naturally handled by the document model in MongoDB. Its purpose is to support graph-oriented features such as user recommendations, location-based recommendations, event recommendations, event attendance, and relationship summaries.

The graph model defines three main node types:

- User
- Event
- Interest.

A **User** node represents an application user who can follow other users, attend events, and be connected to interests. The main predicates for this node are **user_id, username, and location**.

An **Event** node represents an activity that users can attend or receive as a recommendation. Its predicates are **event_id, title, location, and start_date.**

An **Interest** node represents a topic or preference, such as prayer, meditation, or community. Its main predicate is **interest_name.**

The relationships between nodes are modeled using Dgraph UID edges.

- The `follows` edge connects one `User` to another `User`, representing social relationships between users.
- The `attends` edge connects a `User` to an `Event`, representing event participation.
- The `interested_in` edge connects a `User` to an `Interest`, representing the user's preferences.
- The `event_topic` edge connects an `Event` to an `Interest`, representing the topics associated with that event. This structure allows the system to traverse from users to interests and then to other users or events that share the same interests.

The Dgraph schema includes indexes for the predicates that are used as query entry points or filters.

- The **user_id, event_id, and interest_name** predicates use exact indexes because they are used to locate specific nodes.
- The **location** predicate also uses an exact index because location-based recommendations filter users and events by city.
- The **username** and **title** predicates use term indexes to support text-based lookup behavior.
- The **start_date** predicate uses a day index because event queries filter future events by date.

Reverse traversal is explicitly considered in the graph design. The relationship predicates `follows`, `attends`, `interested_in`, and `event_topic` are defined with `@reverse`, which allows queries to traverse relationships in the opposite direction.

For example, the system can count how many users follow a specific user using `count(~follows)`, and it can count how many users attend an event using `count(~attends)`.

Reverse traversal is also used for recommendations: after finding the interests of the current user, the query can traverse `~interested_in` to find other users with the same interests, or `~event_topic` to find events related to those interests.

This graph model supports several query use cases.

- The user recommendation query starts from the logged-in user, follows the `interested_in` relationship to that user's interests, and then uses reverse traversal to find other users connected to the same interests.
- The location-based recommendation query finds users in the same city while excluding the current user and users already followed.
- The local events query returns future events in the same location as the current user.
- The event recommendation query starts from the user's interests and traverses to events that share those topics.
- The graph summary query uses aggregation to return follower counts and attendee counts through reverse edges.

The design is query-driven because each node type, predicate, index, and relationship exists to support a specific graph query required by the application. Users, events, and interests are modeled as nodes instead of embedded attributes because they participate in multiple relationships and are reused across different traversals.

Interests are especially important as connector nodes: they allow the system to recommend both users and events through shared preferences. Dgraph is therefore appropriate for this part of the system because the main operations depend on traversing relationships, excluding already-connected nodes, using reverse edges, and calculating relationship-based counts.

The design was query driven , so the access patterns are thought on the most common queries and Functional Requirements. Each query is mapped to the database whose data model and access patterns minimize the cost of that query. Within each database, the schema, the indexes, and (in Cassandra) the partition and clustering keys are chosen to match those queries directly.

At a high level, the request path is the following:

```
CLI command
   ↓
main.py parser
   ↓
database client file
   ↓
Falcon resource (Mongo, Dgraph, Chroma)
   or direct module call (Cassandra)
   ↓
database model / query
   ↓
database response
   ↓
terminal output
```

User-facing commands (MongoDB, Dgraph, ChromaDB) are exposed through the Falcon HTTP API. Cassandra commands are CLI-only because they represent analytics operations and do not need to be reached from a mobile or web client, but only for admin users that need this metrics to make data driven decisions.

[Docker services running]

```
(.venv) mrmariano@MacBook-Pro-Mariano Team % make start
docker compose up -d
[+] up 4/4
 ✔ Network team_default              Created
 ✔ Container final-project-mongodb   Started
 ✔ Container final-project-cassandra Started
 ✔ Container final-project-dgraph    Started
Waiting for Cassandra to accept CQL connections...
Cassandra not ready yet (1/30).
Cassandra not ready yet (2/30).
Cassandra is ready.
.venv/bin/python main.py seed
Creating MongoDB indexes...
Seeding MongoDB...
MongoDB seed loaded: 5 users, 2 content items
Seeding Dgraph...
Dgraph schema installed
Dgraph seed data loaded
Seeding ChromaDB...
Warning: You are sending unauthenticated requests to the HF Hub.
Loading weights: 100%|
ChromaDB collection ready
Batches: 100%|
ChromaDB seed content loaded (5 documents)
Seed complete. Demo login: demo@mail.com / 1234
.venv/bin/python -m uvicorn main:app --reload --host 0.0.0.0 --po
INFO:     Will watch for changes in these directories: ['/Users/m
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to
INFO:     Started reloader process [20624] using WatchFiles
INFO:     Started server process [20626]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

```
(.venv) mrmariano@MacBook-Pro-Mariano Team % docker compose ps
NAME                     IMAGE                     COMMAND                  SERVICE     CREATED          STATUS
final-project-cassandra  cassandra:4.1             "docker-entrypoint.s…"   cassandra   10 minutes ago   Up 10 minutes (healthy)
final-project-dgraph     dgraph/standalone:v23.1.1 "/run.sh"                dgraph      10 minutes ago   Up 10 minutes
final-project-mongodb    mongo:7                   "docker-entrypoint.s…"   mongodb     10 minutes ago   Up 10 minutes
```

Command:

```
make start
docker compose ps
```

What this means:

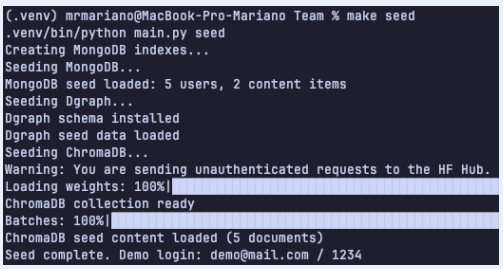
- The make start invocation in the terminal.
- The docker compose ps output listing the running services: mongodb, dgraph, and cassandra, all in a healthy or running state.
- API/backend startup logs from make start.

Rubric Evidence:

Database engine setup and system deployment; Solution Implementation — Connection to database engines; Docker Compose orchestration of multiple NoSQL paradigms.

Figure 1. Docker Compose services for MongoDB, Dgraph, and Cassandra running, with the API started by make start.

[Seed step initializing all four databases]



Command:

`python3 main.py seed ( Done via make.file with make seed )`

The screenshot shows:

- The seed command being executed in the terminal.
- Log lines confirming that MongoDB indexes are created.
- Log lines confirming that the Dgraph schema is installed and seed data is loaded.
- Log lines confirming that the ChromaDB collection is created and populated.

Rubric Evidence:

Database schema management and data loading. Solution Design with MongoDB indexes and Dgraph schema.

Figure 2. Single seed command initializing MongoDB, Dgraph, and ChromaDB consistently from the shared seed file.

Data Models Overall

MongoDB — Operational document store

MongoDB holds the operational application data because users, content, notes, comments, likes, and shares fit naturally as JSON-like documents, which gives us the structure and flexibility to expand on attributes or reduce them, when necessary, in the lifetime of our project. Frequently displayed fields (such as a username on a comment, or a content title on a like) are embedded as small denormalized copies; data that can grow without bound is stored in its own collection and referenced by ObjectId just as for example a field of “user” in a document of “note” can be directly embedded because it doesn’t grow without bounds but is not done like so, because of the increasing complexity for mainting consistency.

Collections:

- users — profile, email, password hash, location, preferences, recent likes.
- content — title, type, creator reference, counts, recent interactions.
- commentResource — comments linked to users and content.
- contentLikes — likes linked to users and content.
- internalShareResource — content shared from one user to another.
- externalShareResource — content shared to an external platform.
- notesResource — personal notes linked to users.

Relationships are represented by ObjectId references plus small, embedded summaries of the most-displayed fields. This avoids extra reads on display paths while keeping a single source of truth in the referenced collection.

Cassandra — Analytics store

Cassandra holds analytics events. Its tables are not normalized; instead, they are shaped around the queries that the analytics layer must answer, so as usual, data is duplicated across multiple tables. The same event is written to more than one table so that each query reads from a partition designed for it.

Table	Partition key	Clustering keys	Query supported
activity_by_user	user_id	timestamp DESC, activity_id	Activity history of one user, newest first.
activity_by_day	date	timestamp DESC, activity_id	Daily active users and system-wide daily activity.
activity_by_content	content_id	timestamp DESC, activity_id	Engagement metrics for one content item.

All main reads are driven by the partition key. When additional filtering is needed, such as filtering a user's history by activity type or date, the current implementation first reads the user's partition and then applies the additional filtering in application logic.

Dgraph — Graph store

Dgraph holds the social and discovery graph:

```
(User)-[follows]->(User)
(User)-[interested_in]->(Interest)
(User)-[attends]->(Event)
(Event)-[event_topic]->(Interest)
```

Reverse edges are declared so the graph can be traversed in both directions. They are used in particular for follower and attendee counts:

```
count(~follows)
count(~attends)
```

The user identifier used in Dgraph (user_id) is the same value as the MongoDB _id for the corresponding user. This consistency is guaranteed by the shared seed_data.py and is required so that a user resolved in MongoDB can be located in the graph for recommendations of friends and events.

ChromaDB — Vector store

ChromaDB holds embedded text representations of content items. For each item, the embedded text combines the title, the type, the tags, and the text body. The embedding function is all-MiniLM-L6-v2. Queries are answered by nearest semantic match.

The vector store supports four operations used by the application:

- semantic_search — free-text query, returns nearest content.
- recommend_content — query built from a user's preferences.
- rag_context — returns the retrieved documents to be used as context for an answer.
- rag_answer — composes a local, template-based answer from the retrieved sources. It does not call an external LLM and is labeled as such in the code and in this report.

[MongoDB profile read]

```
(.venv) mrmariano@MacBook-Pro-Mariano Team % python3 main.py login --email demo@mail.com --password 1234
python3 main.py get_profile

User demo logged in successfully
 Id: 6a00ba6fe204703196b479ec
Age: 21
Created At: 2026-05-15T13:17:20.528000
Email: demo@mail.com
Location: Guadalajara
Preferences: prayer
Updated At: 2026-05-15T13:29:16.866000
Username: demo
```

Command:

```
python3 main.py login --email demo@mail.com --password 1234
python3 main.py get_profile
```

The screenshot shows:

- The login command returning a successful login for the demo user.
- The get_profile output showing the document returned from the user’s collection.
- Fields visible in the profile: username, email, location, preferences.

Rubric Evidence:

Solution Design — MongoDB relationships and document model; query-driven modeling (login and profile path).

Figure 3. Login and profile retrieval against the MongoDB user’s collection for the demo account.

[Dgraph graph summary with reverse-edge counts]

```
(.venv) mrmariano@MacBook-Pro-Mariano Team % python3 main.py graph_summary
{
  "users": [
    {
      "user_id": "6a00ba6fe204703196b479ec",
      "username": "demo",
      "follower_count": 0
    },
    {
      "user_id": "6a00ba6fe204703196b479ed",
      "username": "mariano",
      "follower_count": 0
    },
    {
      "user_id": "6a00ba6fe204703196b479ee",
      "username": "german",
      "follower_count": 1
    },
    {
      "user_id": "6a00ba6fe204703196b479ef",
      "username": "ana",
      "follower_count": 1
    },
    {
      "user_id": "6a00ba6fe204703196b479f0",
      "username": "santiago",
      "follower_count": 1
    }
  ],
  "events": [
    {
      "event_id": "e1",
      "title": "Prayer Circle",
      "attendee_count": 1
    },
    {
      "event_id": "e2",
      "title": "Community Service",
      "attendee_count": 1
    },
    {
      "event_id": "e3",
      "title": "Meditation Meetup",
      "attendee_count": 1
    }
  ]
}
```

Command:

`python3 main.py graph_summary`

The screenshot shows:

- A list of users and events returned from Dgraph.
- Follower counts for users, derived from count(~follows).
- Attendee counts for events, derived from count(~attends).

Rubric Evidence:

Solution Design of Dgraph relationships between nodes; reverse edges; graph aggregations (count).

Figure 4. Graph summary showing follower and attendee counts via reverse-edge aggregations in Dgraph.

Queries

The following table maps each frequently executed query to the database that answers it, to the part of the data model that supports it, and to the CLI command used to demonstrate it.

Query	Database	Model element used	Demo command
Login and read profile	MongoDB	users' collection, indexed by email.	python3 main.py login / get_profile
Create and read personal notes	MongoDB	notesResource, indexed by user.user_id.	python3 main.py create_note / get_notes
Content engagement summary	MongoDB	Aggregation joining content, contentLikes, and commentResource.	python3 main.py content_stats
Activity history of one user	Cassandra	activity_by_user partitioned by user_id.	python3 main.py get_activity_history -- user_id <id>
Daily active users	Cassandra	activity_by_day partitioned by date.	python3 main.py get_daily_active_users -- date <YYYY-MM-DD>
Engagement metrics for a content item	Cassandra	activity_by_content partitioned by content_id.	python3 main.py get_content_metrics -- content_id <id>
Trending content	Cassandra	Aggregation over activity_by_content reads.	python3 main.py trending_content
Recommend users by shared interests	Dgraph	Traversal User → interested_in → ~interested_in → User.	python3 main.py recommend_user
Recommend users in the same location	Dgraph	Indexed location filter on User.	python3 main.py recommend_user_loc
Local events	Dgraph	Indexed location filter on Event.	python3 main.py local_events
Recommend events by interest	Dgraph	Traversal User → interested_in → ~event_topic → Event.	python3 main.py recommend_events
Counts of followers and attendees	Dgraph	Reverse edges ~follows, ~attends.	python3 main.py graph_summary

Query	Database	Model element used	Demo command
Semantic search over content	ChromaDB	Embedded content (title + type + tags + text).	python3 main.py semantic_search --query "..."
Preference-based recommendations	ChromaDB	Query built from user preferences.	python3 main.py recommend_content --preferences <p>
RAG context retrieval	ChromaDB	Nearest-document retrieval.	python3 main.py rag_context --query "..."
Local RAG-style answer	ChromaDB	Retrieved sources + template composition (no external LLM).	python3 main.py rag_answer --query "..."

[MongoDB aggregation pipeline (content_stats)]

```
(.venv) mrmariano@MacBook-Pro-Mariano Team % python3 main.py content_stats
Meditation for Anxiety [meditation] creator=mariano likes=0 comments=0 total=0 id=7000ba6fe204703196b479e2
Peaceful Evening Prayer [prayer] creator=demo likes=0 comments=0 total=0 id=7000ba6fe204703196b479e1
```

Command:

`python3 main.py content_stats`

The screenshot shows:

- The content_stats command being executed.
- The aggregated output: for each content item, the title with corresponding like count and comment count.

Figure 5. MongoDB aggregation pipeline producing per-content like and comment counts.

[Dgraph user recommendation by shared interests]

```
(.venv) mrmariano@MacBook-Pro-Mariano Team % python3 main.py recommend_user
[
  {
    "user_id": "6a00ba6fe204703196b479ed",
    "username": "mariano",
    "location": "Guadalajara",
    "follower_count": 0
  },
  {
    "user_id": "6a00ba6fe204703196b479ee",
    "username": "german",
    "location": "Guadalajara",
    "follower_count": 1
  }
]
```

Command:

`python3 main.py recommend_user`

The screenshot shows:

- The traversal result returning users connected to the logged-in user through a shared interested_in path.
- Display fields of the recommended users (e.g., username, location).

Rubric Evidence:

Solution Design — Dgraph relationships and traversal; query-driven modeling (recommendation).

Figure 6. User-to-user recommendation through a two-hop traversal across interest nodes.

[Dgraph event recommendation]

```
(.venv) mrmariano@MacBook-Pro-Mariano Team % python3 main.py recommend_events
[
  {
    "event_id": "e1",
    "title": "Prayer Circle",
    "location": "Guadalajara",
    "start_date": "2026-06-03T18:00:00Z",
    "attendee_count": 1
  }
]
```

Command:

`python3 main.py recommend_events`

The screenshot shows:

- A list of recommended events obtained by traversing User → interested_in → ~event_topic → Event.
- Display fields of the recommended events (e.g., title, start_date, location).

Rubric Evidence:

Solution Design — Graph traversal with multi-hop edges; reverse edges used for event_topic.

Figure 7. Event recommendations built by traversing interests through their reverse event_topic edges.

[Cassandra event ingestion]

```
(.venv) mrmariano@MacBook-Pro-Mariano Team % python3 main.py recommend_events
[
  {
    "event_id": "e1",
    "title": "Prayer Circle",
    "location": "Guadalajara",
    "start_date": "2026-06-03T18:00:00Z",
    "attendee_count": 1
  }
]
(.venv) mrmariano@MacBook-Pro-Mariano Team % python3 main.py log_session --user_id 6a00ba6fe204703196b479ec --event_type login
Session event 'login' logged [activity_id=c7d52799-443a-4fbc-bccc-ccdcd962f316].
(.venv) mrmariano@MacBook-Pro-Mariano Team % python3 main.py log_activity --user_id 6a00ba6fe204703196b479ec --activity_type like --content_id 7000ba6fe204703196b479e1 --metadata demo-like
Activity 'like' logged [activity_id=575c4567-6959-47f1-a4d1-f9ff5f3cbdeb].
```

Command:

```
python3 main.py log_session --user_id 6a00ba6fe204703196b479ec --event_type login
python3 main.py log_activity --user_id 6a00ba6fe204703196b479ec --activity_type like
--content_id 7000ba6fe204703196b479e1 --metadata demo-like
```

The screenshot shows:

- Both commands executing successfully in the terminal.
- Output confirming that the events were written to Cassandra (to activity_by_user, activity_by_day, and activity_by_content).

Rubric Evidence:

Solution Implementation — Data loading into Cassandra; multi-table write path shaped around queries.

Figure 8. Same event written into multiple Cassandra analytics tables in a single ingestion step.

[Cassandra activity history by user (partition-key read)]

```
(.venv) mrmariano@MacBook-Pro-Mariano Team % python3 main.py get_activity_history --user_id 6a00ba6fe204703196b479ec

Activity history for user: 6a00ba6fe204703196b479ec (2 events)
----------------------------------------------------------------
  Timestamp           2026-05-15 19:49:00.820000
  Type                like
  Content ID          7000ba6fe204703196b479e1
  Metadata            demo-like
----------------------------------------------------------------
  Timestamp           2026-05-15 19:48:50.587000
  Type                login
  Content ID          -
  Metadata            -
----------------------------------------------------------------
```

Command:

```
python3 main.py get_activity_history --user_id 6a00ba6fe204703196b479ec
```

The screenshot shows:

- The list of activity rows returned from activity_by_user for the demo user.

Figure 9. Per-user activity history read from activity_by_user using its partition key.

[Cassandra daily activity (partition by date)]

```
(.venv) mrmariano@MacBook-Pro-Mariano Team % python3 main.py get_daily_active_users --date 2026-05-15

Daily Active Users on 2026-05-15: 1
```

Command:

```
python3 main.py get_daily_active_users --date <YYYY-MM-DD>
```

The screenshot shows:

- The daily aggregate read from activity_by_day for the given date.

Figure 10. Daily activity aggregate read from activity_by_day, partitioned by date.

[Cassandra content metrics (partition by content_id)]

```
(.venv) mrmariano@MacBook-Pro-Mariano Team % python3 main.py get_content_metrics --content_id 7000ba6fe204703196b479e1

Engagement metrics for content: 7000ba6fe204703196b479e1
  Total interactions            1
  Breakdown by type:
    like                        1
```

Command:

```
python3 main.py get_content_metrics --content_id 7000ba6fe204703196b479e1
```

The screenshot shows:

- Engagement metrics for the given content item read from activity_by_content.
- Breakdown of activity types for that content item.

Rubric Evidence:

Solution Design — Cassandra tables shaped around queries; partition key by content_id.

Figure 11. Per-content engagement metrics read from activity_by_content.

[ChromaDB semantic search]

```
(.venv) mrmariano@MacBook-Pro-Mariano Team % python3 main.py semantic_search --query "anxiety and peace"
[
  {
    "id": "content_meditation",
    "title": "Meditation for Anxiety",
    "type": "meditation",
    "tags": "meditation,anxiety,peace",
    "distance": 0.426081120967865,
    "document": "Meditation for Anxiety. Tags: meditation,anxiety,peace. A calming meditation for anxiety, stress, breathing, and inner peace."
  },
  {
    "id": "content_hope",
    "title": "Hope in Difficult Times",
    "type": "reflection",
    "tags": "hope,faith,stress",
    "distance": 0.561962366104126,
    "document": "Hope in Difficult Times. Tags: hope,faith,stress. A short reflection about keeping hope and faith during stressful moments."
  },
  {
    "id": "content_prayer",
    "title": "Evening Prayer",
    "type": "prayer",
    "tags": "prayer,peace,family",
    "distance": 0.6386227607727051,
    "document": "Evening Prayer. Tags: prayer,peace,family. A peaceful evening prayer for family, gratitude, and guidance."
  }
]
```

Command:

`python3 main.py semantic_search --query "anxiety and peace"`

The screenshot shows:

- The list of content items returned by ChromaDB ordered by semantic similarity to the query.

Rubric Evidence:

Solution Design — ChromaDB embeddings; query-driven modeling (semantic retrieval).

Figure 12. Semantic search results from ChromaDB using the all-MiniLM-L6-v2 embedding model.

[ChromaDB local template-based RAG answer]

```
(.venv) mrmariano@MacBook-Pro-Mariano Team % python3 main.py rag_answer --query "I need a peaceful prayer for my family" --limit 2
{
  "query": "I need a peaceful prayer for my family",
  "answer": "Local demo RAG answer (template-based, not an external LLM): the closest retrieved content is Evening Prayer, Meditation for Anxiety. Use these sources as the context for answering the question.",
  "sources": [
    {
      "id": "content_prayer",
      "title": "Evening Prayer",
      "type": "prayer",
      "tags": "prayer,peace,family"
    },
    {
      "id": "content_meditation",
      "title": "Meditation for Anxiety",
      "type": "meditation",
      "tags": "meditation,anxiety,peace"
    }
  ],
  "context": "Evening Prayer: Evening Prayer. Tags: prayer,peace,family. A peaceful evening prayer for family, gratitude, and guidance.\n\nMeditation for Anxiety: Meditation for Anxiety. Tags: meditation,anxiety,peace. A calmin
g meditation for anxiety, stress, breathing, and inner peace."
}
```

Command:

`python3 main.py rag_answer --query "I need a peaceful prayer for my family" --limit 2`

The screenshot shows:

- The retrieved sources used as context.
- The locally composed template answer.

- Evidence that the answer is locally composed from sources and does not call an external LLM.

Rubric Evidence:
Solution Design — ChromaDB embeddings connected to a RAG-style workflow; query-driven modeling.

Figure 13. Local RAG-style answer composed from semantically retrieved Chroma documents.

Indexes and Optimizations

Indexes and partitioning choices are not generic. Each one supports a query that the application actually struggles at, so the indexes are chosen for specific reasons such as frequency in querying.

MongoDB indexes

Indexes are created by the seed step in _setup_mongo_indexes. They cover the fields used by operational filters and are non-unique to avoid demo-time failures from pre-existing local data:

- users: email, username, location, preferences.
- content: creator, type, created date.
- commentResource: content reference, user reference.
- contentLikes: user reference, content reference.
- internalShareResource and externalShareResource: user reference, content reference.
- notesResource: user.user_id, created date.

MongoDB aggregation as an optimization

The content_stats command answers the engagement-summary query in a single pipeline using $lookup, $project, $size, and $sort, rather than issuing one read per content item from the client.

Cassandra partition and clustering keys

Each analytics table is partitioned by the query anchor (user_id, date, or content_id) and clustered by timestamp DESC and activity_id, so recent events can be retrieved first without extra sorting. The main optimization is writing the same event into multiple denormalized tables: this increases write cost, but it enables efficient primary-key reads for each analytics query and avoids inefficient patterns such as full scans or ALLOW FILTERING, which usually indicates that the schema was not designed around the query.

Dgraph schema indexes

The schema declares indexes on the fields used by lookups and filters: user_id, username, location, event_id, title, start_date, and interest_name. Edges that need traversal in both

directions like (follows, attends, interested_in, event_topic) are declared with @reverse so that count(~follows) and count(~attends) resolve without a second query.

ChromaDB precomputed embeddings

Content items are embedded once at seed time with all-MiniLM-L6-v2. At query time only the query string is embedded, and the comparison is delegated to Chroma's nearest-neighbor index.

[MongoDB indexes confirmed on key collections]

```
final_project> db.users.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { email: 1 }, name: 'email_1' },
  { v: 2, key: { username: 1 }, name: 'username_1' },
  { v: 2, key: { location: 1 }, name: 'location_1' },
  { v: 2, key: { preferences: 1 }, name: 'preferences_1' }
]
final_project> db.content.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  {
    v: 2,
    key: { 'created_by.user_id': 1 },
    name: 'created_by.user_id_1'
  },
  { v: 2, key: { type: 1 }, name: 'type_1' },
  { v: 2, key: { created_at: 1 }, name: 'created_at_1' }
]
final_project> db.notesResource.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { 'user.user_id': 1 }, name: 'user.user_id_1' },
  { v: 2, key: { created_at: 1 }, name: 'created_at_1' }
]
```

Command:

```
mongosh "mongodb://localhost:27017/religious_app"
db.users.getIndexes()
db.content.getIndexes()
db.notesResource.getIndexes()
```

The screenshot shows:

- The output of getIndexes() for at least the users, content, and notesResource collections.
- The indexes created by the seed step visible in that output.

Rubric Evidence:

Solution Design — MongoDB indexes. Solution Implementation — schema management.

Figure 14. MongoDB indexes installed by the seed step, verified through the Mongo shell.

Conclusion

The project delivers one backend that uses four non-relational databases, each chosen because of how its data needs to be queried, not because of personal preferences. MongoDB carries the operational document workload and answers engagement queries through an aggregation pipeline. Cassandra carries the analytics workload and answers every analytics query by partition key, with the same event written into several query-shaped tables. Dgraph carries the social and discovery graph and supports counts, location filters, and multi-hop recommendations through reverse edges. ChromaDB carries semantic retrieval, with a clearly scoped local template-based answer for the RAG path.

The query-driven design is what holds the four stores together. Each query was identified first via carefully resolving the query according to each requirement, then a database and a schema were picked to make that query cheap. The shared seed step guarantees that the same user identifier exists in MongoDB and Dgraph, which removes the most common source of inconsistency between an operational store and a graph store in this kind of system.

Some limitations remain and are documented honestly rather than hidden. The synchronization of preference removals from MongoDB into Dgraph does not currently delete the corresponding interest edges. Cassandra is populated through explicit analytics CLI commands rather than from every normal application action. The rag_answer command composes a local template-based answer from retrieved Chroma documents and does not call an external Large Language Model. These boundaries are part of the project scope and not gaps in design.

Sources of Information

The following technical references were consulted during the design and implementation of the project.

- Falcon Framework documentation. https://falcon.readthedocs.io/ (Accessed: 15 Apr 2026)
- Docker Compose reference. https://docs.docker.com/compose/ (Accessed: 12 May 2026)
- MongoDB documentation — Indexes. https://www.mongodb.com/docs/manual/indexes/ (Accessed: 24 Apr 2026)
- MongoDB documentation — Aggregation pipeline, $lookup, $project, $size. https://www.mongodb.com/docs/manual/core/aggregation-pipeline/ (Accessed: 02 May 2026)
- Apache Cassandra documentation — Data modeling and primary keys. https://cassandra.apache.org/doc/latest/ (Accessed: 26 Apr 2026)
- Apache Cassandra documentation — CQL data definition. https://cassandra.apache.org/doc/latest/ https://cassandra.apache.org/doc/latest/cassandra/cql/ddl.html (Accessed: 30 Apr 2026)
- Dgraph documentation — Schema, indexes, and reverse edges. https://docs.dgraph.io / (Accessed: 27 Apr 2026)
- Dgraph documentation — DQL queries and aggregations. https://dgraph.io/docs/query-language/ (Accessed: 29 Apr 2026)
- Chroma documentation — Collections and queries. https://docs.trychroma.com/ (Accessed: 06 May 2026)
- Sentence-Transformers — all-MiniLM-L6-v2 model card. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2 (Accessed: 07 May 2026)

Appendix

This appendix contains supporting material that did not fit into the previous sections; the reproducible demo flow, and the explicit known limitations of the implementation.

Final Demo Flow

The recommended demo order, designed to show each database paradigm in turn. Run from the repository root, in two terminals: terminal A keeps the services running; terminal B runs the CLI commands.

Terminal A — services

```
make start
```

Terminal B — demo

```
# Seed once
make seed

# Login
python3 main.py login --email demo@mail.com --password 1234

# MongoDB
python3 main.py get_profile
python3 main.py content_stats

# Dgraph
python3 main.py graph_summary
python3 main.py recommend_user
python3 main.py recommend_user_loc
python3 main.py local_events
python3 main.py recommend_events

# ChromaDB
python3 main.py semantic_search --query "anxiety and peace"
python3 main.py recommend_content --preferences prayer --limit 3
python3 main.py rag_context --query "I need a peaceful prayer for my family" --limit 2
python3 main.py rag_answer  --query "I need a peaceful prayer for my family" --limit 2

# Cassandra
python3 main.py log_session --user_id 6a00ba6fe204703196b479ec --event_type login
python3 main.py log_activity --user_id 6a00ba6fe204703196b479ec --activity_type like --
content_id 7000ba6fe204703196b479e1 --metadata demo-like
python3 main.py get_activity_history --user_id 6a00ba6fe204703196b479ec
python3 main.py get_content_metrics  --content_id 7000ba6fe204703196b479e1
python3 main.py get_daily_active_users --date "$(date +%F)"
```

Demo credentials:

```
email:    demo@mail.com
password: 1234
```

The same _id is used for the demo user in MongoDB and as user_id in Dgraph. This consistency is established by seed_data.py and is what makes the cross-database flow possible.

Known Limitations

The boundaries of the implementation are documented explicitly so that the demonstration and the report describe only what the system actually does.

- MongoDB-to-Dgraph synchronization updates graph-relevant user fields but does not delete interested_in edges when a preference is removed.
- Cassandra analytics events are written by explicit demo commands (log_session, log_activity), not by every normal application action.
- rag_answer composes a local template-based answer from retrieved Chroma documents. It does not call an external Large Language Model.
- The CLI commands update, add_pref, and rem_pref are interactive.
- The Falcon API must be running (make start) for MongoDB-, Dgraph-, and Chroma-facing CLI commands.